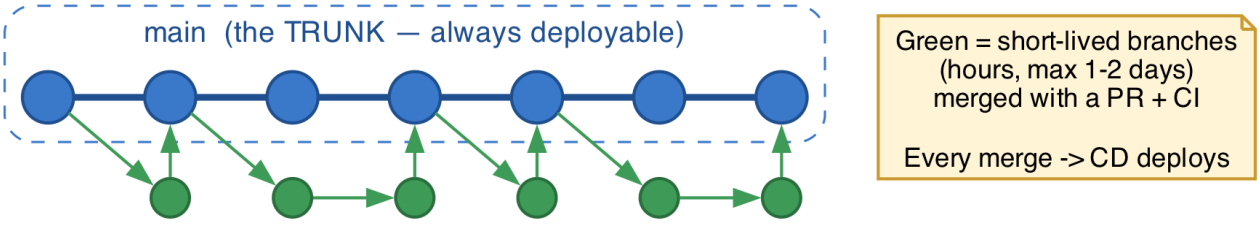
Git & GitHub
for DevOps / Platform Engineers

Trunk-Based Development, from zero to interview-ready



One trunk (main), many tiny branches, merged every day, deployed every merge.

How to use this guide

Read one part, then do the matching lab in **luul-trunk-lab/LAB.md**. Green boxes are answers you can say in an interview. Blue boxes show how it works at a real company.

Contents

1. Git basics: what, why, the 3 areas
2. The commands you use every day
3. Trunk-based development (and why not GitFlow)
4. The daily workflow, step by step
5. Feature flags: deploy is not release
6. Merge, rebase and conflicts
7. CI/CD with GitHub Actions
8. The platform engineer's GitHub setup
9. Incidents: rollback, revert, fix forward
10. Releases, tags and release branches
11. GitOps
12. The practice project: map of the labs
13. Interview questions and answers
14. Your interview story (STAR)
15. One-page cheat sheet

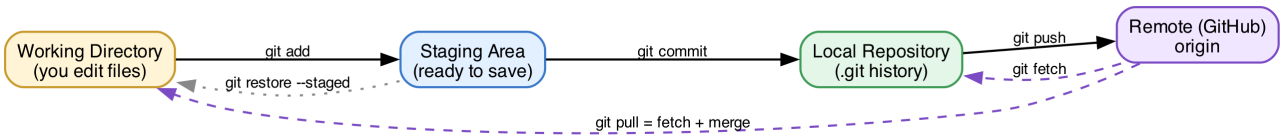
1. Git basics

Git is a program on your computer that remembers every version of your files. Each save is called a **commit**. You can always go back to an older commit, see who changed what, and work in parallel with other people.

GitHub is a website that stores Git repositories online. It adds team features: **Pull Requests** (code review), **Actions** (CI/CD), **Issues**, **Packages** (container registry), security scanning and permissions.

> **Say this in the interview**
>
> Git is a **distributed** version control system: every developer has the full history locally. GitHub is a hosting and collaboration platform on top of Git, with pull requests, Actions for CI/CD, and branch protection.

The three areas (+ the remote)



Files move left to right with add, commit, push. They come back with fetch and pull.

Words you must know

Word	Simple meaning
Repository (repo)	A project folder tracked by Git (the hidden .git folder holds the history)
Commit	A snapshot of changes + a message + author + a unique ID (SHA, like a1b2c3d)
Branch	A movable name pointing to a commit, so you can work without touching main
main / trunk	The shared branch. In trunk-based development it is always deployable
HEAD	Where you are right now (usually the tip of your current branch)
Remote / origin	The copy on GitHub. 'origin' is its default name
Clone / Fetch / Pull / Push	Download repo / download new commits / fetch + merge / upload commits
Pull Request (PR)	A request to merge your branch into main, with review and CI checks
Merge / Rebase / Squash	Join branches / replay commits on top / combine many commits into one
Tag	A permanent name for a commit, e.g. v1.2.0 for a release
Conflict	Two changes touched the same lines; Git needs a human to choose

2. The commands you use every day

Goal	Command
One-time setup	`git config --global user.name "Name"` `git config --global user.email you@mail.com`
Start / copy a repo	`git init` or `git clone git@github.com:org/repo.git`
What changed?	`git status`, `git diff`, `git diff --staged`
Save	`git add -A` then `git commit -m "feat: add login"`
History	`git log --oneline --graph --all`
Branches	`git switch -c feat/x` (create) , `git switch main` , `git branch -d feat/x`
Sync	`git fetch` , `git pull --rebase` , `git push -u origin feat/x`
Update my branch	`git fetch && git rebase origin/main`
Undo file edits	`git restore file` ; unstage: `git restore --staged file`
Fix last commit	`git commit --amend`
Undo a pushed commit (safe)	`git revert <sha>`
Undo local commits	`git reset --soft HEAD~1` (keep changes) / `--hard` (delete!)
Park work	`git stash` , `git stash pop`
Copy one commit	`git cherry-pick <sha>`
Find lost work	`git reflog`
Who changed this line?	`git blame file` ; find bad commit: `git bisect`
Release	`git tag -a v1.2.0 -m msg && git push origin v1.2.0`
GitHub CLI	`gh pr create --fill` , `gh pr checks --watch` , `gh pr merge --squash`

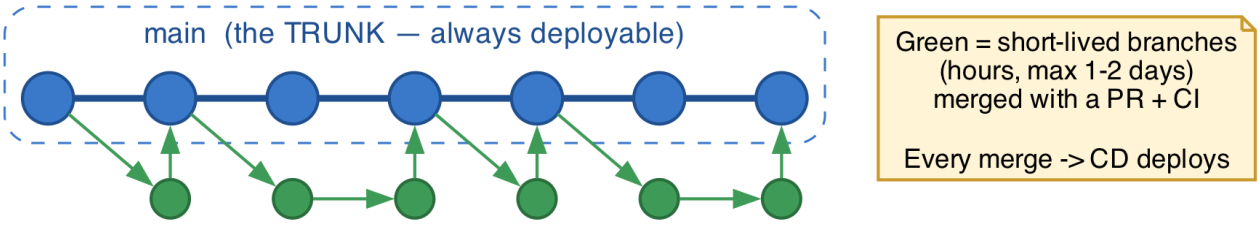
Good commit messages (Conventional Commits)

```
feat: add health endpoint           # new feature  -> MINOR version
fix: correct nginx port in helm chart # bug fix      -> PATCH version
chore: bump terraform provider        # maintenance
docs: explain rollback process
feat!: drop v1 API                    # ! = breaking -> MAJOR version
```

Small commits with clear messages make code review, `git revert` and release notes easy.

3. Trunk-based development

Trunk-based development (TBD) means all developers merge **small** changes into one shared branch, the **trunk** (`main`), **at least once a day**. Branches live for hours, not weeks. The trunk is always green (tests pass) and always ready to deploy.

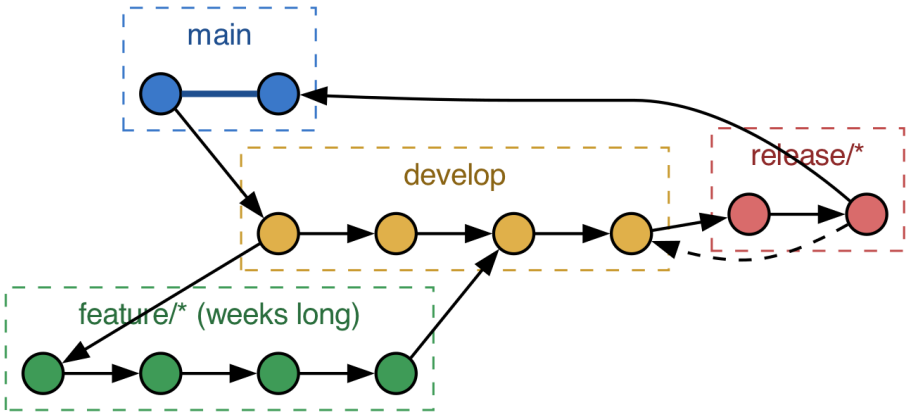


Trunk-based: short green branches merge back quickly. Each merge triggers CI/CD.

The rules

- **One trunk:** `main`. No long-lived `develop` branch.
- **Short-lived branches:** less than 1-2 days, ideally a few hours. Small PRs (about 200 lines or less).
- **Trunk is always releasable:** required CI checks protect it. Broken main = everybody stops and fixes it.
- **Unfinished work is hidden behind feature flags**, not kept on a long branch.
- **Build once, deploy many:** the same image (tagged with the commit SHA) goes dev, then staging, then prod.
- **Fix forward or revert on trunk.** Hotfixes go through the same pipeline.
- **Release branches are optional**, cut from trunk only when you must support an old version. Fixes are cherry-picked from trunk.

Compare: GitFlow



GitFlow: main + develop + long feature branches + release branches. More merging, more conflicts.

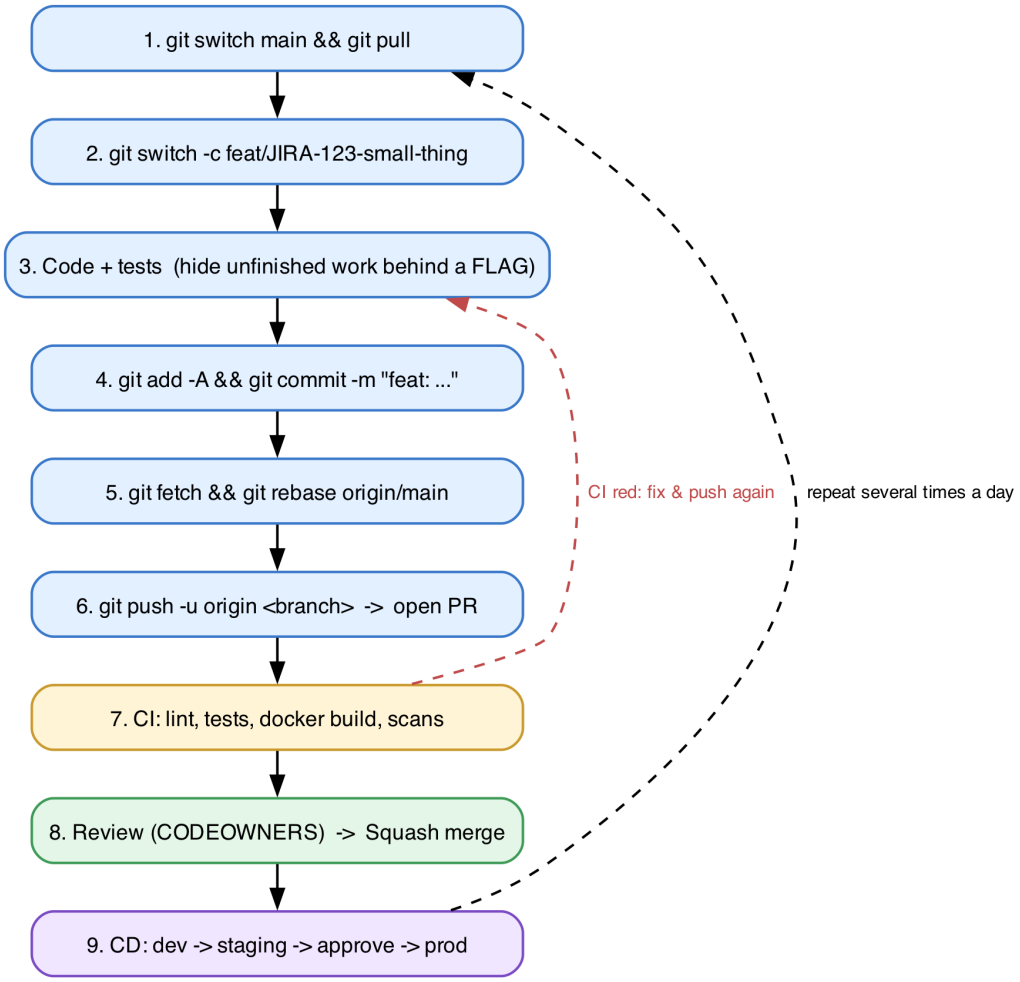
	Trunk-based	GitFlow
Long-lived branches	Only main	main + develop (+ release, hotfix)

	Trunk-based	GitFlow
Feature branch life	Hours to 1-2 days	Days to weeks
Merge conflicts	Small and rare	Big and painful ('merge hell')
Unfinished features	Feature flags	Stay on the branch
Release speed	Many times a day (continuous delivery)	Scheduled releases
Needs	Strong automated tests + CI/CD + flags	Release manager, more process
Best for	Web services, SaaS, cloud platforms, DevOps teams	Versioned products, slow release cycles

Say this in the interview

We use trunk-based development. Everyone merges small, reviewed changes into main every day. Branches live less than a day or two, unfinished features are hidden behind feature flags, and every merge goes through CI and is automatically deployed. It's what DORA research links to high-performing teams: more frequent deploys, lower change-failure rate and faster recovery. We chose it over GitFlow because long-lived branches cause merge conflicts and slow down delivery.

4. The daily workflow



The loop you repeat several times a day.

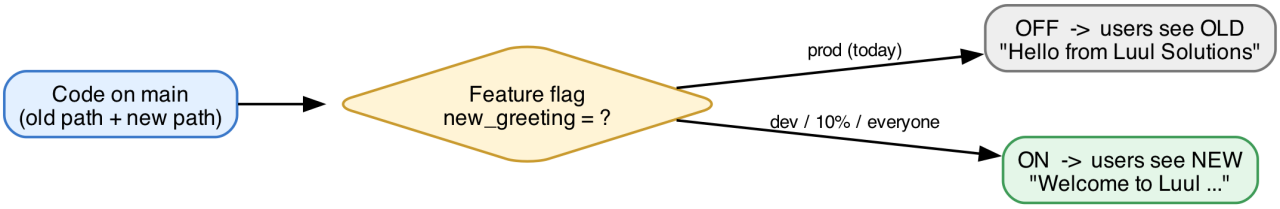
```
git switch main && git pull                    # 1 latest trunk
git switch -c feat/PLAT-123-add-alerts           # 2 short-lived branch
# ... edit + test ...                            # 3 small change
git add -A && git commit -m "feat: add CPU alert"   # 4
git fetch && git rebase origin/main              # 5 stay fresh
git push -u origin feat/PLAT-123-add-alerts      # 6
gh pr create --fill && gh pr checks --watch      # 7 CI
gh pr merge --squash                             # 8 merge (after review)
git switch main && git pull                      # back to 1
```

At work (Luul Solutions style)

Branch names often include the ticket: `feat/PLAT-123-...`, `fix/...`, `chore/...`. Many teams turn on **auto-merge**: the PR merges itself once review and CI are green. A merge queue tests PRs together so main never breaks when many people merge at the same time.

5. Feature flags: deploy is not release

The biggest question about trunk-based development is: *"How do you merge a feature that is not finished?"* The answer is a **feature flag** (also called a feature toggle). The new code is merged and deployed, but it is switched OFF. When it's ready, you turn it ON without a new deploy.



Same code everywhere. The flag decides what users see.

In the practice project (`app/main.py`):

```
def greeting(flags):
    if flags.get("new_greeting"):          # flag from config/flags.json
        return "Welcome to Luul Solutions platform!"
    return "Hello from Luul Solutions"      # old behaviour stays the default

# Per environment override:  FLAG_NEW_GREETING=true make run
```

Other ways to merge unfinished work

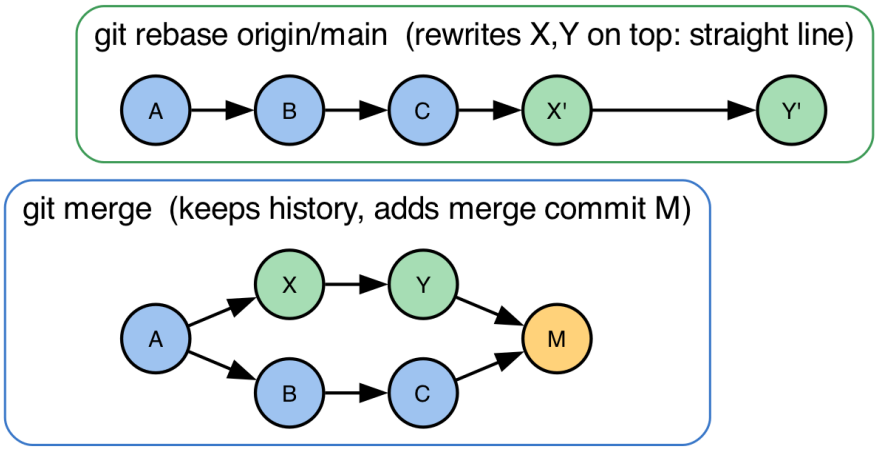
- **Branch by abstraction:** put an interface in front of the old code, build the new code behind it, switch over, delete the old code.
- **Dark launch:** the new code runs but its result is not shown to users (to test performance).
- **Canary / percentage rollout:** turn the flag on for 1%, then 10%, then 100% of users.

Flag hygiene

- Remove a flag and its old code path soon after the feature is at 100%. Old flags are technical debt.
- Tools: LaunchDarkly, Unleash, Flagsmith, AWS AppConfig, OpenFeature (standard API), or simple config/env vars.

> **Say this in the interview**
>
> **Deployment** puts code in production. **Release** exposes it to users. With feature flags we separate them: we merge and deploy daily with the flag off, then release gradually by turning it on. If something breaks we turn the flag off in seconds, with no rollback deploy.

6. Merge, rebase and conflicts



Merge keeps both lines and adds M. Rebase replays X and Y on top of C (new IDs X', Y').

	Merge	Rebase	Squash merge (on GitHub)
History	True history, with merge commits	Clean straight line	One commit per PR on main
Rewrites commits?	No	Yes (new SHAs)	Yes (combines them)
Use on	Shared branches	Your OWN branch only	Merging PRs into trunk

Golden rule: never rebase or force-push a branch other people use (like `main`). On your own branch after a rebase, use `git push --force-with-lease`. It refuses to overwrite work you haven't seen.

Resolving a conflict

```
$ git rebase origin/main
CONFLICT (content): Merge conflict in app/main.py

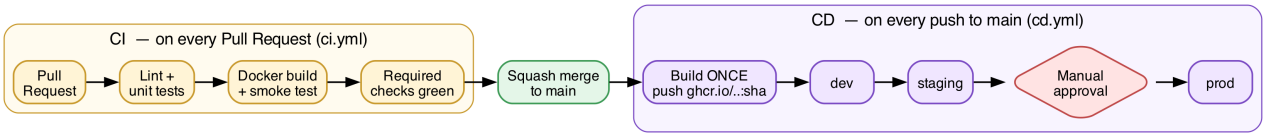
<<<<<<< HEAD              # what is already on main
VERSION = "2.0.0"
=======
VERSION = "1.1.0"         # your change
>>>>>>> feat/bump

# 1. edit the file: keep the right code, delete the 3 marker lines
# 2. git add app/main.py
# 3. git rebase --continue      (or git commit if it was a merge)
# 4. make test, then git push --force-with-lease
# Panic button: git rebase --abort  /  git merge --abort
```

> **Say this in the interview**
>
> Trunk-based development keeps conflicts small because branches are short and I rebase on main often. When I get a conflict, I read both sides, talk to the other author if unsure, resolve it, run tests locally and push. I never just pick 'mine' blindly.

7. CI/CD with GitHub Actions

CI (Continuous Integration): every PR is automatically tested so trunk stays green. **CD (Continuous Delivery/Deployment)**: every commit on trunk is automatically built and deployed. Delivery means it is *ready* for prod, with a human approval. Deployment means it goes to prod fully automatically.



The pipeline in the practice project: ci.yml on PRs, cd.yml on main.

GitHub Actions vocabulary

Word	Meaning
Workflow	A YAML file in .github/workflows/
Trigger (on:)	When it runs: pull_request, push, tags, schedule, workflow_dispatch (manual button)
Job	A group of steps on one runner. Jobs run in parallel unless you use needs:
Step	One command (run:) or a reusable action (uses: actions/checkout@v4)
Runner	The machine: GitHub-hosted (ubuntu-latest) or self-hosted (your own, e.g. in Kubernetes with ARC)
Secrets / vars	${{ secrets.X }} encrypted values. GITHUB_TOKEN is created automatically for each run
Environment	dev/staging/prod with their own secrets, required reviewers and wait timers
permissions:	What GITHUB_TOKEN may do. Give the least privilege (contents: read)
concurrency:	Stops two deploys from running at the same time
OIDC	Log in to AWS/Azure/GCP with short-lived tokens, so no cloud keys are stored in GitHub
Reusable workflow	workflow_call: one shared pipeline for 100 repos (platform team owns it)
Matrix	Run the same job for several versions/OS, e.g. python 3.11 and 3.12

The CI file from the project (simplified)

```yaml
name: CI
on:
  pull_request:
    branches: [main]
permissions:
  contents: read
jobs:
  test:
    name: Lint & unit tests          # this name is a REQUIRED check
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: make test
  build:
    name: Docker build & smoke test
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t app:ci .
      - run: ./scripts/smoke-test.sh app:ci
```

The CD idea

```yaml
on: { push: { branches: [main] } }
jobs:
  build:          # test, then build ONE image: ghcr.io/owner/repo:<sha>, push it
  deploy-dev:     { needs: build,          environment: dev }
  deploy-staging: { needs: deploy-dev,     environment: staging }
  deploy-prod:    { needs: deploy-staging, environment: prod }   # waits for approval
```

> **Say this in the interview**
>
> On every PR, CI runs lint, unit tests, a Docker build, a smoke test and security scans, and branch protection makes them required. On merge to main, CD builds the image once, tags it with the commit SHA, pushes it to the registry and promotes the same artifact through dev and staging to prod. Prod is a GitHub environment with required reviewers. We authenticate to the cloud with OIDC, so there are no long-lived keys.

8. The platform engineer's GitHub setup

As a DevOps/Platform engineer you don't only *use* Git. You **design the guardrails** that let every team ship safely.

Guardrail	What it does	Where
Branch protection / Rulesets	Require a PR, required status checks, 1-2 approvals, linear history, no force-push, no deletion	Settings > Branches / Rules (rulesets can apply to the whole org)
CODEOWNERS	Auto-requests review from the owning team, e.g. /terraform/ @org/platform	.github/CODEOWNERS
Squash merge only + auto-delete branches	One clean commit per PR on trunk, no stale branches	Settings > General
Merge queue	Tests PRs in order against the latest main so trunk never breaks	Rulesets
Environments	dev/staging/prod secrets, required reviewers, branch restrictions	Settings > Environments
Secrets + OIDC	No passwords in code. Short-lived cloud credentials	Settings > Secrets; cloud IAM trust
Dependabot	PRs to update vulnerable dependencies and actions	.github/dependabot.yml
Secret scanning + push protection	Blocks pushes that contain keys/tokens	Settings > Code security
CodeQL / Trivy / tfsec	Code, image and IaC security scans in CI	Workflow steps
Pin actions	Use a full commit SHA for 3rd-party actions (supply-chain safety)	uses: org/action@<sha>
Signed commits	Proves who made the commit	Ruleset: require signed commits
Repo as code	Manage repos, teams and protections with Terraform (github provider)	Platform repo

The practice project does the basics for you: `./scripts/setup-github.sh owner/repo`

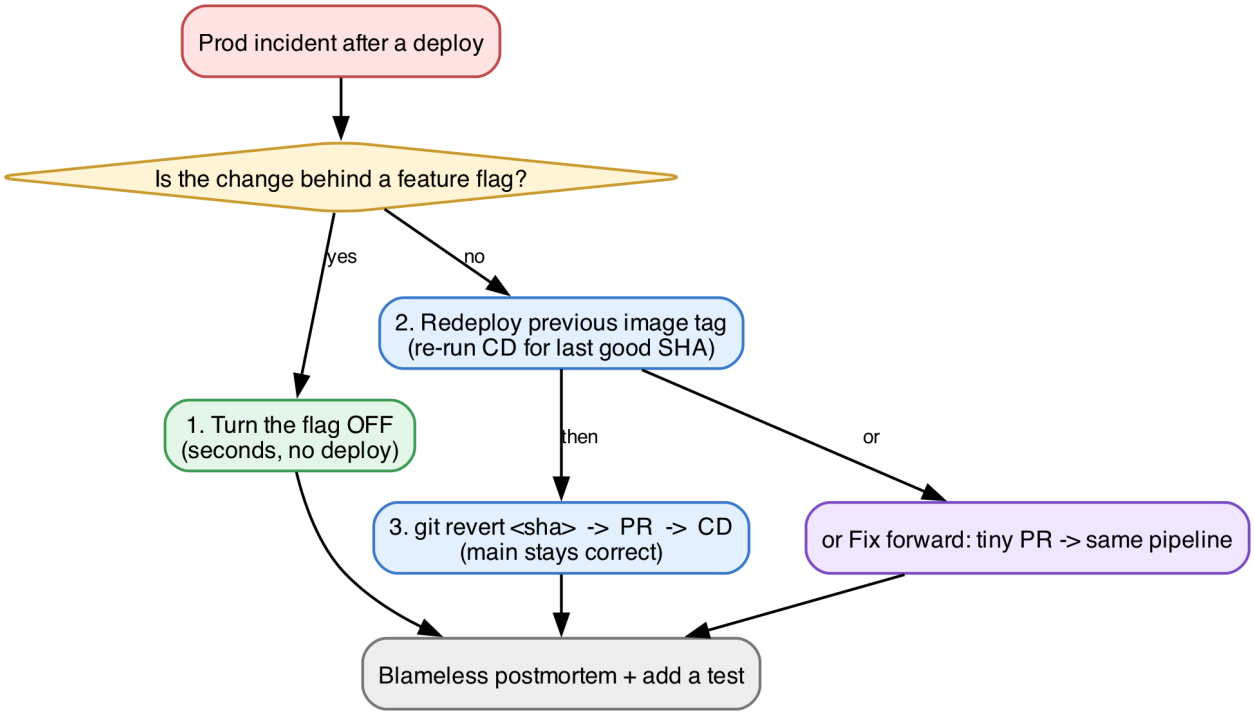
.gitignore and secrets

```
.env
*.pem
*.key
.terraform/
terraform.tfstate*
__pycache__/
node_modules/
```

A secret was pushed. What now?

1) **Revoke/rotate it immediately**. It's already public, and bots scan GitHub within seconds. 2) Remove it from history (git filter-repo or BFG) and force-push with the team's agreement. 3) Check logs for misuse. 4) Turn on push protection so it can't happen again.

9. Incidents: rollback, revert, fix forward



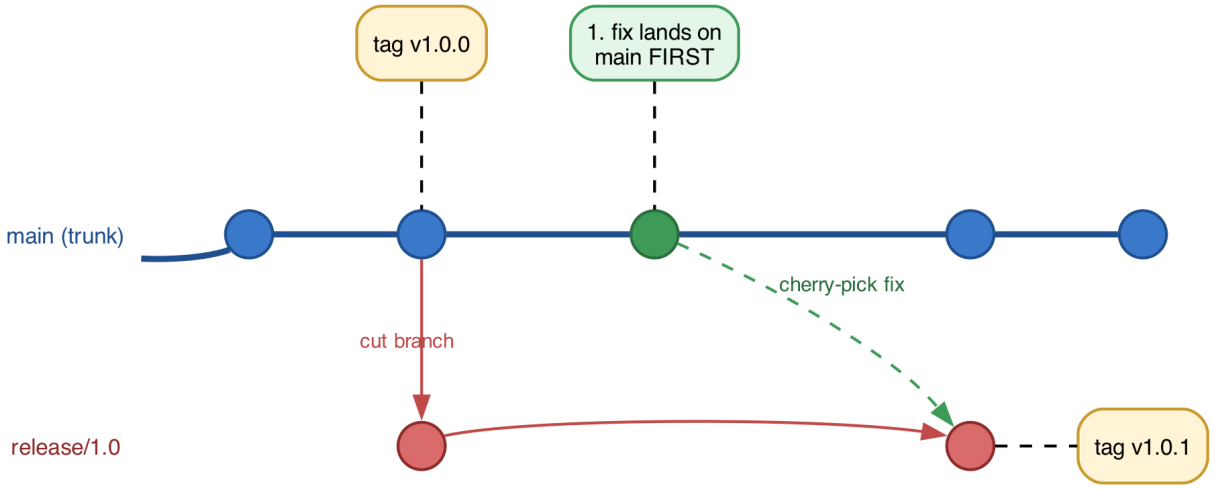
Decision path when a deploy breaks production.

Command	What it does	Safe on main?
`git revert <sha>`	Adds a NEW commit that undoes an old one. History kept.	YES. Use this
`git reset --hard <sha>`	Moves the branch back and deletes commits after it.	NO (local only)
`git reset --soft HEAD~1`	Undoes the commit, keeps the changes staged.	Local only
`git bisect`	Binary search through history to find the commit that broke something.	Yes (read only)

> **Say this in the interview**
>
> The fastest fix is to turn off the feature flag. If there's no flag, we redeploy the previous image, which is easy because every image is tagged with its commit SHA. Then we `git revert` the bad commit on main so trunk is correct again, or fix forward with a tiny PR through the same pipeline. Afterwards we write a blameless postmortem and add a test so the bug can't come back.

10. Releases, tags and release branches

Web services in trunk-based teams are often deployed continuously, and tags just mark versions. A **release branch** is needed only when you must support an older version (mobile apps, on-prem products, or a slow release train).



Release from trunk: the fix lands on main FIRST, then it is cherry-picked to release/1.0.

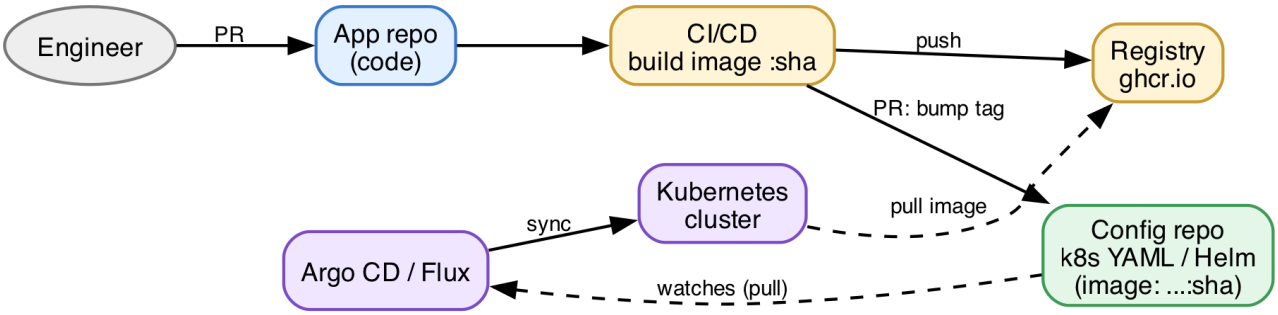
```
git tag -a v1.0.0 -m "Release 1.0.0" && git push origin v1.0.0  # runs release.yml
git switch -c release/1.0 v1.0.0 && git push -u origin release/1.0
git cherry-pick f00ba12                    # the fix already merged on main
git tag v1.0.1 && git push origin release/1.0 v1.0.1
```

Semantic versioning MAJOR.MINOR.PATCH: breaking change, new feature, bug fix. Tools like release-please or semantic-release read Conventional Commits and create tags and changelogs automatically.

> **Say this in the interview**
>
> We release from trunk. Normally a tag on main triggers the release workflow. If we have to maintain an old version, we cut a release branch from the tag and cherry-pick fixes into it. Fixes always go to trunk first, and we never merge a release branch back.

11. GitOps

GitOps applies the same Git workflow to infrastructure and deployments. The desired state of the cluster lives in Git. A controller (Argo CD or Flux) runs **inside** the cluster, watches the repo, and keeps the cluster matching Git. This is called **pull-based** deployment.



CI builds the image; a PR bumps the tag in the config repo; Argo CD syncs it to Kubernetes.

- **Every change is a PR:** reviewed, audited, and easy to revert.
- **Rollback** = `git revert` the tag bump.
- **Drift detection:** if someone runs `kubectl edit` in prod, Argo CD shows it as OutOfSync and can undo it.
- **Same for Terraform:** PR, then `terraform plan` shown as a PR comment (Atlantis / Terraform Cloud / Actions), then merge, then `apply`.

> **Say this in the interview**
>
> With GitOps, Git is the single source of truth. CI builds and pushes the image, then opens a PR that updates the image tag in our config repo. When it's merged, Argo CD syncs the cluster. No one has kubectl write access to prod, every change is audited, and rollback is a git revert.

12. The practice project: luul-trunk-lab

Folder: `~/wezva-ai-driven-bootcamp/git-github/luul-trunk-lab`. Open `LAB.md` and do the labs in order.

Lab	You practise	Guide part
0	Create the GitHub repo, protect main, environments, squash-only	8
1	The daily trunk-based loop: branch, PR, CI, squash merge, CD to prod with approval	3, 4, 7
2	Break CI on purpose: see that branch protection blocks the merge	7, 8
3	Feature flag: merge unfinished work, enable it per environment	5
4	Rebase your branch on a newer main, force-with-lease	6
5	Create and resolve a merge conflict	6
6	Roll back with git revert	9
7	Hotfix by fixing forward	9
8	Tag a release, release branch + cherry-pick	10
9	git stash for context switching	2
10	Recover lost commits with git reflog	2
11	GitOps: change the image tag in k8s/deployment.yaml	11

Project files

```
luul-trunk-lab/
|-- app/main.py                 # web service + feature flag
|-- config/flags.json           # flags (new_greeting: false)
|-- tests/test_main.py          # unit tests (make test)
|-- Dockerfile                  # non-root image with healthcheck
|-- Makefile                    # test / run / build / smoke
|-- k8s/deployment.yaml         # Kubernetes / GitOps manifest
|-- scripts/
|   |-- setup-github.sh         # platform setup (one time)
|   |-- protect-main.sh         # branch protection via gh api
|   |-- smoke-test.sh           # start container, check /health
|   `-- deploy.sh               # simulated deploy
`-- .github/
    |-- workflows/ci.yml        # PR checks
    |-- workflows/cd.yml        # main -> dev -> staging -> prod
    |-- workflows/release.yml   # tag v*.*.* -> GitHub Release
    |-- CODEOWNERS
    `-- pull_request_template.md
```

13. Interview questions and answers

Q1. What is the difference between Git and GitHub?

Git is the version control tool. GitHub hosts Git repos and adds PRs, Actions, permissions and security features.

Q2. What is trunk-based development?

Everyone merges small changes into one branch (main) at least daily. Branches are short-lived, main is always deployable, unfinished work is behind feature flags, and every merge is built and deployed by CI/CD.

Q3. Why trunk-based instead of GitFlow?

Fewer and smaller merge conflicts, faster feedback, continuous delivery, and a simpler process. GitFlow suits versioned products with scheduled releases. It slows down cloud services.

Q4. How do you merge a feature that is not finished?

Behind a feature flag (off by default), or using branch by abstraction. The code ships dark and is released later.

Q5. What makes trunk-based development safe?

Required CI checks, good automated tests, small PRs with review, feature flags, a merge queue, and fast rollback.

Q6. git fetch vs git pull?

Fetch downloads commits without changing your files. Pull = fetch + merge (or rebase with --rebase).

Q7. Merge vs rebase?

Merge joins histories with a merge commit and doesn't rewrite anything. Rebase replays your commits on top of another branch, giving a linear history but new SHAs. Only rebase your own branches.

Q8. What is a squash merge and why use it?

It combines all PR commits into one commit on main. Trunk stays clean, and one PR = one commit is easy to revert.

Q9. git reset vs git revert?

Reset moves the branch pointer and rewrites history (local use). Revert adds a new undo commit, which is safe on shared branches.

Q10. What is HEAD? What is a detached HEAD?

HEAD points to your current commit/branch. Detached means you checked out a commit, not a branch. Create a branch to keep your work.

Q11. How do you resolve a merge conflict?

Open the file, choose the correct code, remove the markers, git add, continue the merge/rebase, run tests, push.

Q12. What is git stash?

It temporarily saves uncommitted changes so you can switch branches. Get them back with git stash pop.

Q13. What is cherry-pick?

It copies one commit to another branch, e.g. a fix from main to a release branch.

Q14. How do you find which commit broke something?

git bisect: a binary search between a good and a bad commit. git blame shows who last changed a line.

Q15. How do you recover a deleted branch or lost commit?

git reflog to find the SHA, then git switch -c rescue .

Q16. How do you protect the main branch?

Branch protection/rulesets: required PR + approvals, required status checks, linear history, no force-push/delete, CODEOWNERS, merge queue.

Q17. Describe your CI/CD pipeline.

PR: lint, tests, build, scans (required checks). Merge: build one image tagged with the SHA, push to registry, deploy dev, then staging, then prod with approval (or a GitOps sync).

Q18. How do you handle secrets in pipelines?

GitHub secrets per environment, OIDC to the cloud (no static keys), least-privilege GITHUB_TOKEN, secret scanning with push protection, Vault/Secrets Manager at runtime.

Q19. A secret was committed. What do you do?

Rotate it first, then purge history with git filter-repo, check for misuse, and enable push protection.

Q20. How do you roll back a bad deploy?

Turn off the flag, redeploy the previous SHA image, git revert on main, or fix forward. Then a postmortem.

Q21. What is GitOps?

Git holds the desired state. A controller like Argo CD or Flux pulls and syncs it to the cluster. PR = deploy, revert = rollback, and drift is detected.

Q22. How do hotfixes work in trunk-based development?

As a small PR to main through the normal pipeline. If a release branch exists, cherry-pick the fix to it.

Q23. What is a monorepo vs polyrepo?

One repo for many services (shared tooling, atomic changes, needs path-filtered CI) vs one repo per service (independent, but harder to keep consistent).

Q24. How would you speed up a slow pipeline?

Cache dependencies and Docker layers, run jobs in parallel, use path filters, split tests, use bigger or self-hosted runners, and cancel outdated runs with concurrency.

Q25. What are DORA metrics?

Deployment frequency, lead time for changes, change failure rate, and time to restore. Trunk-based development improves all four.

14. Your interview story (STAR)

Interviewers love real examples. Build the practice project, then tell it like this:

	Example answer
Situation	Our team used long feature branches. Merges were painful and releases happened every two weeks.
Task	As the platform engineer, I had to make delivery faster and safer.
Action	I moved the repo to trunk-based development: protected main with required CI checks and CODEOWNERS, allowed squash merge only, built a GitHub Actions pipeline that builds one image per commit and promotes it dev, then staging, then prod with environment approvals, added feature flags for unfinished work, and used OIDC instead of stored cloud keys.
Result	We went from deploying every two weeks to several times a day. Merge conflicts almost disappeared, and rollback takes minutes (flag off or redeploy the previous SHA).

> **Be honest**
>
> If it was a personal or lab project, say so: *"In my lab project I built..."*. Showing the GitHub repo with green pipelines is strong proof.

15. One-page cheat sheet

```
# DAILY TRUNK LOOP
git switch main && git pull
git switch -c feat/TICKET-short-name
git add -A && git commit -m "feat: ..."
git fetch && git rebase origin/main
git push -u origin HEAD
gh pr create --fill && gh pr checks --watch && gh pr merge --squash

# LOOK
git status | git diff | git log --oneline --graph --all | git show <sha>

# UNDO
git restore <file>             # throw away edits
git restore --staged <file>    # unstage
git commit --amend             # fix last commit (not pushed)
git reset --soft HEAD~1        # undo last commit, keep changes
git revert <sha>               # undo a pushed commit (SAFE)
git reflog                     # find anything you "lost"

# CONFLICTS
edit file -> git add <file> -> git rebase --continue   (or --abort)

# OTHER
git stash / git stash pop
git cherry-pick <sha>
git bisect start / bad / good <sha>
git tag -a v1.2.0 -m "..." && git push origin v1.2.0
git push --force-with-lease    # only on YOUR branch, never main
```

Trunk-based in one line each	
Branch life	Hours, at most 1-2 days
Unfinished work	Feature flag OFF
Main	Always green, always deployable, protected
Merge style	Squash, linear history
Artifact	Build once (tag = SHA), promote dev, then staging, then prod
Hotfix	Normal small PR to main (+ cherry-pick to release branch if one exists)
Rollback	Flag off, then previous image, then git revert